

Breathe ESG Tech Intern Assignment – Solution Guide

This document provides an accurate and defensible solution approach for the Breathe ESG internship assignment. The focus is on realistic data ingestion, normalization, auditability, analyst workflows, and practical tradeoffs.

1. Recommended Tech Stack

Backend: Django + Django REST Framework + PostgreSQL

Frontend: React + TypeScript + Material UI

Queue/Async: Celery + Redis

Deployment: Railway or Render

Storage: AWS S3 or Cloudinary for uploaded files

Authentication: JWT authentication using SimpleJWT

2. System Architecture

The system should follow a pipeline architecture: Data ingestion from multiple enterprise sources Raw source preservation Normalization into a canonical ESG schema Validation and anomaly detection Analyst review and approval workflow Audit locking and immutable history tracking Each uploaded file or API pull generates an ingestion batch. Rows are stored first as raw records before transformation.

3. SAP Fuel & Procurement Handling

Chosen Format: CSV flat-file export from SAP ECC/S4HANA.

This is realistic because many enterprises export SAP data into CSV or Excel for downstream reporting. The prototype should support: German or English column names Plant codes Mixed units (liters, gallons, kilograms) Cost center mapping Vendor-based procurement categorization **Example Mapping:** Diesel purchase → Scope 1 Purchased electricity equipment → Scope 3 Fuel transport invoices → Scope 3 upstream transport

4. Utility Electricity Data

Chosen Input: CSV export from a utility portal.

Reason: Most facilities teams export monthly CSV files PDF parsing is possible but adds complexity CSV is realistic for enterprise workflows Required fields: Meter ID Billing period start/end Consumption (kWh) Tariff type Site location The system should normalize all energy values to kWh and split billing periods across months when needed.

5. Corporate Travel Data

Chosen Source: Concur-style REST API payload export.

Handled categories: Flights Hotels Rail Ground transport Important logic: Airport codes converted to distances using lookup tables Flight class impacts emission factor Hotel nights converted using regional average factors Ground transport classified by vehicle type

6. Core Data Model

Recommended entities: Organization Facility DataSource IngestionBatch RawRecord NormalizedEmissionRecord ReviewDecision AuditLog
Key design principles: Multi-tenancy via organization foreign keys Immutable raw data storage Versioned normalized records Full source traceability

7. Suggested Database Tables

Table	Purpose
organizations	Multi-tenant client isolation
facilities	Site and meter association
data_sources	SAP / Utility / Travel metadata
ingestion_batches	Track upload batches
raw_records	Store untouched imported rows
normalized_records	Canonical ESG records
review_actions	Analyst approval/rejection history
audit_logs	Immutable audit trail

8. Analyst Dashboard Features

The dashboard should include: Batch upload status Failed row counts Normalization warnings Suspicious value detection Review and approval actions Audit lock status
Examples of suspicious rows: Negative electricity consumption Fuel quantities 10x above historical average Unknown airport codes Missing units

9. API Design

Recommended REST endpoints: POST /api/uploads/sap POST /api/uploads/utility POST /api/uploads/travel GET /api/review/queue POST /api/review/approve POST /api/review/reject

10. Auditability Strategy

Auditability is critical in ESG systems. The system should: Never overwrite raw source data Store original uploaded files Track who edited each record Track timestamps for every action Lock approved records from further editing

11. Tradeoffs to Mention

Good tradeoffs to explicitly document: No OCR support for scanned utility PDFs No live SAP integration in MVP No automatic emission-factor updates These are reasonable because the assignment only allows 4 days.

12. Deployment Recommendation

Recommended deployment: Frontend on Vercel Backend on Railway or Render PostgreSQL managed instance Redis for async queues Use Docker for consistent deployment.

13. Final Evaluation Tips

During review interviews: Defend every architectural choice Explain why raw data is immutable Show realistic sample data Demonstrate understanding of ESG workflows Be honest about limitations The evaluators care more about reasoning than feature count.

This solution balances realism, scalability, auditability, and implementation feasibility within the assignment timeline. The strongest submissions will demonstrate thoughtful modeling decisions and realistic handling of enterprise ESG data challenges.